

Black Box Reverse Engineering Assignment

This assignment consists of a single Python module:

- `oracle.py` – A compiled "black box" pricing engine. It exposes a function named `quote()` that calculates the shipping cost for an order. The implementation is intentionally hidden, so the only way to understand how it works is by calling the function with different inputs and observing the results.

Your goal is to determine the pricing rules used by the engine and reproduce its behavior as accurately as possible. Think of the oracle as a third-party service whose implementation is unavailable. You can query it with any valid inputs, but you cannot rely on reading or modifying its implementation.

The pricing algorithm is deterministic. Given the same inputs, it will always return the same output. All of the information required to solve this assignment can be discovered by designing and running experiments against the `quote()` function.

API

```
from oracle import quote, queries_used, reset_counter

price = quote(
    weight_kg=2.0,          # float, 0.1 to 100
    distance_km=100.0,      # float, 1 to 5000
    category="standard",    # one of: standard, electronics, fragile,
                           #         books, clothing, food
    express=False,          # bool
    coupon="",              # str, e.g. "WELCOME10" or ""
)

queries_used()    # Returns the number of quote() calls made.
reset_counter()   # Resets the counter to zero.
```

If `quote(2.0, 100, "standard")` returns a price, your setup is working correctly.

Getting Started

`oracle.py` is a Python module, **not** a standalone program. You should import it into your own Python code and call the `quote()` function to investigate its behavior.

Place `oracle.py` and any scripts you write in the same directory. You can use any development environment you prefer, including VS Code, PyCharm, Jupyter Notebook, or a terminal.

For example, create a file named `probes.py` in the same directory as `oracle.py`:

```
from oracle import quote

print(quote(2.0, 100, "standard"))
```

Then run:

```
python3 probes.py
```

You may also use an interactive Python REPL:

```
python3
```

```
>>> from oracle import quote
>>> quote(2.0, 100, "standard")
```

As your investigation progresses, you'll likely want to automate your experiments by writing small scripts that call `quote()` with different inputs and compare the results.

No third-party libraries are required. A standard Python 3 installation is sufficient.

Tasks

1. Reimplement the pricing engine in a file named `my_quote.py`. Your implementation should expose a function with the same signature as `quote()` and return the same result for any valid input, **without importing or calling** `oracle.py`.
2. Write `SPEC.md` describing every pricing rule you discovered. Explain each rule in plain English and include at least one example query that demonstrates it.
3. Submit any scripts or code you used to investigate the oracle.
4. Include a short summary of your investigation process (5–10 bullet points), including any hypotheses that turned out to be incorrect.

Notes

- You may use any language or tools, including AI assistants.
- The assignment is intended to be solved by experimentation. Your submission should reflect your own investigation and reasoning.
- Do not inspect or reverse engineer the implementation of `oracle.py`. Treat it as an external service whose behavior must be inferred from its outputs.